

SRG091X&SRG093X 系列

Linux 外设用户指导

短距离产品

版本：1.0.0

日期：2026-08-18

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司

上海市松江区泗泾镇外婆泾路 8 号 邮编：201601

电话：+86 21 5108 6236

邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<https://www.quectel.com.cn/contact>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：

<https://www.quectel.com.cn/contact?tab=t> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持您的产品设计。您须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。您知悉并同意，移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档，增加、修改或重述后的文档对您具有约束力。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，任何形式的购买都不可被视为授予除正常的非独家、免版税的产品使用许可之外的任何许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法、侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他软硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何明示、暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2026，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2026.

文档历史

修订记录

版本	日期	作者	变更表述
-	2026-08-13	Lyman Zhu	文档创建
1.0.0	2026-08-18	Lyman Zhu	临时版本

目录

文档历史	3
目录	4
表格索引	5
1 引言	6
2 环境准备	7
3 GPIO	9
3.1. 设备树配置	9
3.2. GPIO 测试	10
4 I2C	11
5 SPI	13
5.1. 设备树配置	13
5.2. SPI 测试	14
6 以太网	15
7 USB	17
7.1. 设备树配置	17
7.2. USB 工作模式介绍	18
8 ADC	20
8.1. 设备树配置	20
8.2. 获取 ADC 电压	20
9 CAN	22
10 附录 术语缩写	23

表格索引

表 1: 硬件环境	7
表 2: 术语缩写	23

1 引言

SRG091X 系列和 SRG093X 系列是移远通信推出的高度集成的模块，支持多种外设接口。本文以 SRG093X-W 为例，介绍如何在 SR-IMXM-EVB 底板上配置 GPIO、I2C、SPI、ENET、USB、ADC 和 CAN 等外设接口。

2 环境准备

表 1：硬件环境

硬件名称	数量
移远通信 SRG093X-W-TE-A	2
移远通信 SR-IMXM-EVB	2
USB Type-C 线	2
5V 电源	2

备注

使用两套 SR-IMXM-EVB + SRG093X-W-TE-A 配置 CAN 接口，以下简称“开发板 A”和“开发板 B”。

请根据下图中的标注配置跳线帽和杜邦线。

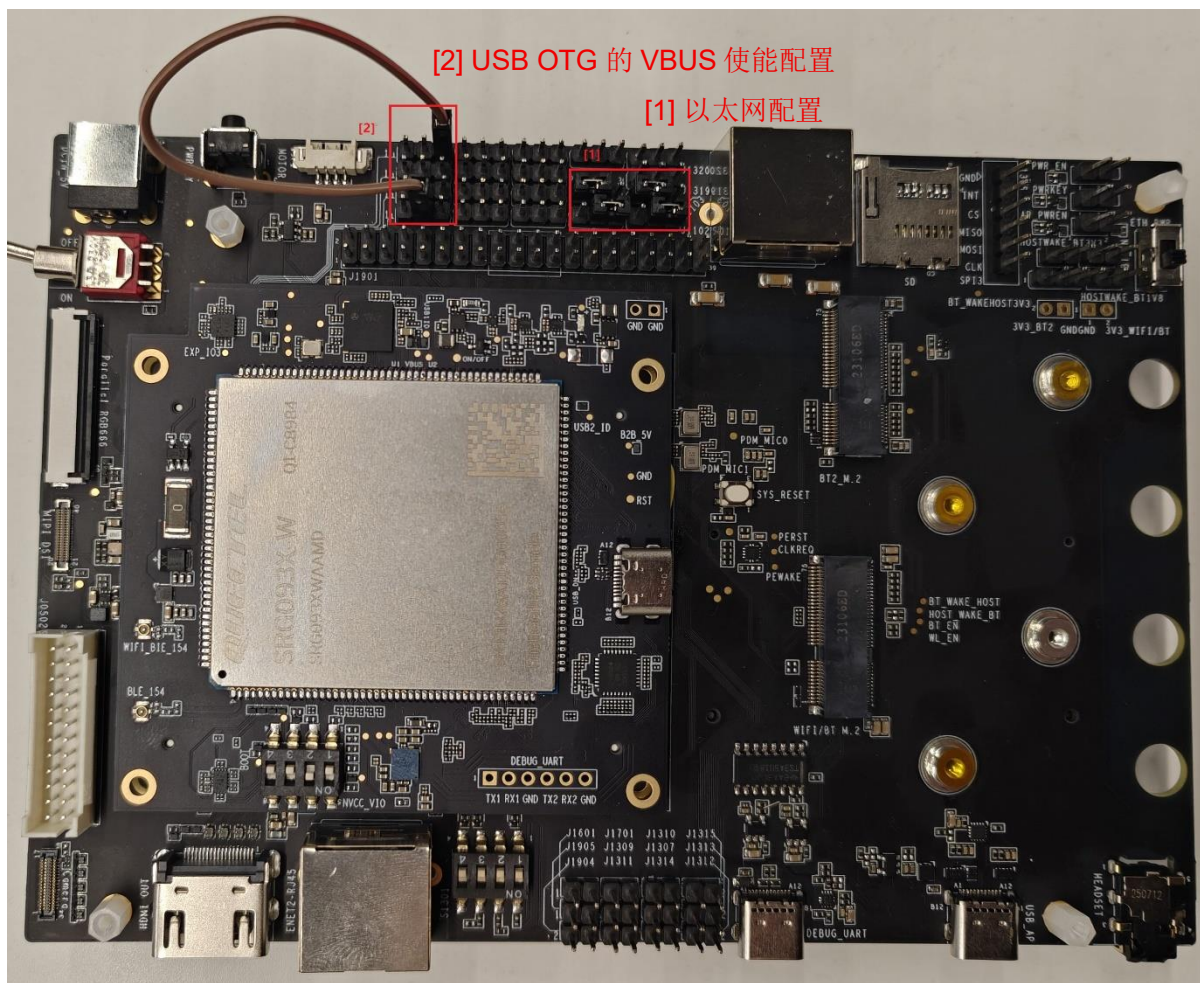


图 1: 配置 SR-IMXM-EVB

3 GPIO

本章介绍如何使用 `libgpiod` 命令行工具控制 GPIO 引脚，以及如何在内核设备树中配置 GPIO。

3.1. 设备树配置

在内核源码的 `arch/arm64/boot/dts/freescale` 目录中，可以找到引脚功能定义文件 `imx93-pinctrl.h` 或 `imx91-pinctrl.h`。例如，如果需要配置 GPIO_IO12（焊盘名称），可在 `imx93-pinctrl.h` 文件中找到一组相关定义。

```
#define MX93_PAD_GPIO_IO12__GPIO2_IO12      0x0040 0x01F0 0x0000 0x0 0x0
#define MX93_PAD_GPIO_IO12__TPM3_CH2        0x0040 0x01F0 0x0000 0x1 0x0
#define MX93_PAD_GPIO_IO12__PDM_BIT_STREAM02 0x0040 0x01F0 0x0440 0x2 0x0
#define MX93_PAD_GPIO_IO12__MEDIAMIX_DISP_DATA08 0x0040 0x01F0 0x0000 0x3 0x0
#define MX93_PAD_GPIO_IO12__LPSPI8_PCS0      0x0040 0x01F0 0x0000 0x4 0x0
#define MX93_PAD_GPIO_IO12__LPUART8_TX       0x0040 0x01F0 0x0000 0x5 0x0
#define MX93_PAD_GPIO_IO12__LPI2C8_SDA       0x0040 0x01F0 0x0404 0x16 0x1
#define MX93_PAD_GPIO_IO12__SAI3_RX_SYNC     0x0040 0x01F0 0x0450 0x7 0x0
```

需要根据实际硬件配置，参照上述说明在设备树中添加相应配置。

```
&iomuxc {
    pinctrl_sample: sample {
        fsl,pins = <
            /* add your GPIO definitions */
        >;
    };
};
```

备注

有关 `pinctrl` 的详细配置说明，请参阅内核源码 `Documentation/devicetree/bindings/pinctrl` 目录下的 `fsl,imx93-pinctrl.yaml` 文件。

3.2. GPIO 测试

libgpiod 项目提供了底层 C 库、高级语言绑定以及配套工具，用于在 Linux 系统中操作 GPIO 线路。它用于替代已被 Linux 内核弃用的传统 GPIO sysfs 接口。

目前提供以下命令行工具：

- **gpiodetect**: 列出系统中的所有 GPIO 芯片，包括其名称、标签以及所包含的 GPIO 线路数量。
- **gpioinfo**: 列出各 GPIO 线路的信息，包括所属的 GPIO 芯片、偏移量、名称和方向。
- **gpioget**: 读取指定 GPIO 线路的电平值。
- **gpioset**: 设置指定 GPIO 线路的电平值，并保持该线路的状态，直到进程终止（被强制结束或正常退出）。

可使用 gpiodetect 工具获取系统中可用的 gpiochip 设备。

```
root@quectel-srg093xw:~# gpiodetect
gpiochip0 [43810000.gpio] (32 lines)
gpiochip1 [43820000.gpio] (32 lines)
...
```

可使用 gpioinfo 工具读取指定 gpiochip 上所有 GPIO 线路的信息。

```
root@quectel-srg093xw:~# gpioinfo -c 1
gpiochip1 - 32 lines:
    line 0:      unnamed          input active-low consumer=cd
    line 1:      unnamed          input
    line 2:      unnamed          input
...
```

可使用 gpioset 和 gpioget 工具分别设置和读取指定 GPIO 线路的电平值（以设置和读取 gpiochip0:25 电平值为例）。

```
root@quectel-srg093xw:~# gpioset -t 0 -c gpiochip0 25=1
root@quectel-srg093xw:~# gpioset -t 0 -c gpiochip0 25=0
root@quectel-srg093xw:~# gpioget -c gpiochip0 25
"25"=inactive
```

备注

使用 gpioset 和 gpioget 工具操作的引脚，需要在设备树中配置为 GPIO 功能。

4 I2C

大多数 I2C 设备由 Linux 内核驱动进行管理。这些驱动通过设备树配置或平台数据绑定到对应的 I2C 总线。用户只需调用相应的内核子系统接口，无需直接处理底层 I2C 通信。

I2C 工具包提供了用于底层访问 I2C 设备的实用工具：

- **i2cdetect**: 扫描 I2C 总线上的设备，并以表格形式列出在指定总线上检测到的设备。
- **i2cdump**: 查看可通过 I2C 总线访问的寄存器。
- **i2cset**: 设置可通过 I2C 总线访问的寄存器。
- **i2cget**: 读取可通过 I2C 或 SMBus 访问的寄存器。

执行 **ls -l /dev/i2c-*** 查看可用的 I2C 总线列表。

```
root@quectel-srg093xw:~# ls -l /dev/i2c-*
crw----- 1 root root 89, 0 Feb 27 21:04 /dev/i2c-0
crw----- 1 root root 89, 1 Feb 27 21:04 /dev/i2c-1
crw----- 1 root root 89, 2 Feb 27 21:04 /dev/i2c-2
```

执行 **i2cdetect -y -r 0** 检测 i2c-0 总线上的设备。

```
root@quectel-srg093xw:~# i2cdetect -y -r 0
 0 1 2 3 4 5 6 7 8 9 a b c d e f
00: -----
10: ----- UU --- UU ---
20: -----
30: -- 32 ----- UU ---
40: ----- 4a -----
50: -----
60: -----
70: -----
```

执行 **i2cdump -r 0-0xf 0 0x4a b** 读取 I2C 设备的寄存器内容。

```
root@quectel-srg093xw:~# i2cdump -r 0-0xf 0 0x4a b
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will probe file /dev/i2c-0, address 0x4a, mode byte
Probe range limited to 0x00-0x0f.
Continue? [Y/n] y
 0 1 2 3 4 5 6 7 8 9 a b c d e f 0123456789abcdef
00: 00 06 00 00 00 00 20 00 09 03 00 00 00 00 00 62 .?.... ??.....b
```

执行以下命令读取寄存器值、修改其值，并通过再次读取验证修改结果。

- 1) 执行 **i2cget 0 0x4a 9 w** 读取寄存器 0x09 的值。

```
root@quectel-srg093xw:~# i2cget 0 0x4a 9 w
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will read from device file /dev/i2c-0, chip address 0x4a, data address
0x09, using read word data.
Continue? [Y/n] y
0x0003
```

- 2) 执行 **i2cset 0 0x4a 9 0x0103 w**，将寄存器 0x09 的值修改为 0x0103。

```
root@quectel-srg093xw:~# i2cset 0 0x4a 9 0x0103 w
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will write to device file /dev/i2c-0, chip address 0x4a,
data address 0x09, data 0x103, mode word.
Continue? [Y/n] y
```

- 3) 执行 **i2cget 0 0x4a 9 w** 再次读取该寄存器的值，以验证修改结果。

```
root@quectel-srg093xw:~# i2cget 0 0x4a 9 w
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will read from device file /dev/i2c-0, chip address 0x4a, data address
0x09, using read word data.
Continue? [Y/n] y
0x0103
```

5 SPI

本章介绍如何在模块上启用 SPI 和 SPIDEV。

5.1. 设备树配置

本章节以 lpspi3 为例，介绍如何配置设备树。

以下内容展示如何在设备树中启用 SPI 并配置 SPIDEV。

```
&lpspi3{
    fsl,spi-num-chipselects = <1>;
    pinctrl-names = "default";
    pinctrl-0 = <&pinctrl_lpspi3>, <&pinctrl_lpspi3_int>;
    cs-gpios = <&gpio2 8 GPIO_ACTIVE_LOW>;
    status = "okay";

    spidev0: spi@0 {
        reg = <0>;
        compatible = "lwn,bk4";
        spi-max-frequency = <1000000>;
    };

    &iomuxc {
        pinctrl_lpspi3: lpspi3grp {
            fsl,pins = <
                MX93_PAD_GPIO_IO08__GPIO2_IO08    0x3fe
                MX93_PAD_GPIO_IO09__LPSPI3_SIN     0x3fe
                MX93_PAD_GPIO_IO10__LPSPI3_SOUT    0x3fe
                MX93_PAD_GPIO_IO11__LPSPI3_SCK     0x3fe
            >;
        };

        pinctrl_lpspi3_int: lpspi3int {
            fsl,pins = <
```

```

        MX93_PAD_GPIO_IO12__GPIO2_IO12    0x31e
    >;
};
};

```

pinctrl 配置必须根据实际硬件设计进行调整。上述示例对应的硬件设计如下：

```

SPI_CS : GPIO_IO08
SPI_MISO : GPIO_IO09
SPI_MOSI : GPIO_IO10
SPI_CLK : GPIO_IO11
SPI_INT : GPIO_IO12

```

5.2. SPI 测试

执行 `ls /dev/spidev*`，列出全部可访问的 SPIDEV 设备节点。

```

root@quectel-srg093xw:~# ls /dev/spidev*
/dev/spidev0.0

```

以下示例展示如何使用 `spidev_test` 工具通过 `spidev` 接口执行读取操作。请确保将设备路径替换为模块实际对应的设备路径。

```

root@quectel-srg093xw:~# spidev_test -D /dev/spidev0.0 -v
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 kHz)
TX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF F0 0D |.....@.....|
RX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
FF FF F0 0D |.....@.....|

```

备注

进行 SPI 回环测试时，将 MISO 和 MOSI 引脚短接，以验证 SPI 的收发功能。

6 以太网

SRG091X 系列和 SRG093X 系列模块内部集成了两个以太网控制器：快速以太网控制器（FEC）和以太网服务质量控制器（ENET_QOS）。以太网控制器由 U-Boot 或 Linux 中相应的设备驱动启用和配置。本章介绍如何进行以太网性能测试。

以下分别介绍 PC 作为客户端、SRG093X-W 模块作为服务器，以及二者角色互换时的性能测试方法。

在 SRG093X-W 模块上启动服务器：

```
root@quectel-srg093xw:~# iperf3 -s -i 1
-----
Server listening on 5201 (test #1)
-----
Accepted connection from 192.168.11.10, port 53717
[ 5] local 192.168.11.65 port 5201 connected to 192.168.11.10 port 53718
[ ID] Interval      Transfer   Bitrate
[ 5] 0.00-1.00  sec  86.5 MBytes  725 Mb/s/sec
[ 5] 1.00-2.00  sec  86.6 MBytes  727 Mb/s/sec
[ 5] 2.00-3.00  sec  83.4 MBytes  699 Mb/s/sec
[ 5] 3.00-4.00  sec  99.1 MBytes  832 Mb/s/sec
[ 5] 4.00-5.00  sec  97.9 MBytes  821 Mb/s/sec
[ 5] 5.00-6.00  sec  92.6 MBytes  777 Mb/s/sec
[ 5] 6.00-7.00  sec  70.9 MBytes  595 Mb/s/sec
[ 5] 7.00-8.00  sec  97.5 MBytes  818 Mb/s/sec
[ 5] 8.00-9.00  sec  98.8 MBytes  828 Mb/s/sec
[ 5] 9.00-10.00 sec  92.6 MBytes  777 Mb/s/sec
[ 5] 10.00-10.01 sec  1.00 MBytes  888 Mb/s/sec
-----
[ ID] Interval      Transfer   Bitrate
[ 5] 0.00-10.01  sec  907 MBytes  760 Mb/s/sec      receiver
```

在 PC 上启动客户端：

```
D:\iperf-3.1.3-win64>iperf3.exe -c 192.168.11.65 -i 1 -t 10
Connecting to host 192.168.11.65, port 5201
[ 4] local 192.168.11.10 port 53718 connected to 192.168.11.65 port 5201
[ ID] Interval      Transfer   Bandwidth
[ 4] 0.00-1.00  sec  87.5 MBytes  734 Mb/s/sec
[ 4] 1.00-2.00  sec  85.9 MBytes  720 Mb/s/sec
```

```
[ 4] 2.00-3.01 sec 83.5 MBytes 696 Mbits/sec
[ 4] 3.01-4.01 sec 99.8 MBytes 833 Mbits/sec
[ 4] 4.01-5.00 sec 97.8 MBytes 829 Mbits/sec
[ 4] 5.00-6.00 sec 92.6 MBytes 776 Mbits/sec
[ 4] 6.00-7.00 sec 71.2 MBytes 598 Mbits/sec
[ 4] 7.00-8.00 sec 97.2 MBytes 816 Mbits/sec
[ 4] 8.00-9.00 sec 99.1 MBytes 829 Mbits/sec
[ 4] 9.00-10.00 sec 92.5 MBytes 778 Mbits/sec
```

[ID]	Interval	Transfer	Bandwidth	
[4]	0.00-10.00	sec 907 MBytes	761 Mbits/sec	sender
[4]	0.00-10.00	sec 907 MBytes	761 Mbits/sec	receiver

7 USB

SRG091X 系列和 SRG093X 系列模块支持两路 USB 2.0 接口：USB1 连接到 EVB 上的 Type-C 接口（丝印：USB_AP），用于固件烧录和 USB-OTG 功能；USB2 连接到 M.2 接口，用于 USB Wi-Fi 通信。

7.1. 设备树配置

如下方设备树所示，USB1 配置为 OTG 模式，USB2 配置为主机（Host）模式。

```
&usbotg1 {
    dr_mode = "otg";
    usb-role-switch;
    disable-over-current;
    samsung,picophy-pre-emp-curr-control = <3>;
    samsung,picophy-dc-vol-level-adjust = <7>;
    status = "okay";

    port {
        usb1_drd_sw: endpoint {
            remote-endpoint = <&typec1_dr_sw>;
        };
    };
};

&usbotg2 {
    dr_mode = "host";
    disable-over-current;
    samsung,picophy-pre-emp-curr-control = <3>;
    samsung,picophy-dc-vol-level-adjust = <7>;
    status = "okay";
};

&lpi2c3 {
    ptn5150: ptn5150@1d {
        compatible = "nxp,ptn5150";
        reg = <0x1d>;
        interrupt-parent = <&gpio3>;
    };
};
```

```

interrupts = <27 IRQ_TYPE_LEVEL_LOW>;
pinctrl-names = "default";
pinctrl-0 = <&pinctrl_ptn5150>;

vbus-gpios = <&gpio2 3 GPIO_ACTIVE_HIGH>;
int-gpios = <&gpio3 27 GPIO_ACTIVE_LOW>;

port {
    typec1_dr_sw: endpoint {
        remote-endpoint = <&usb1_drd_sw>;
    };
};
};

```

usb0tg1 中的 port 节点引用了 USB Type-C CC 逻辑芯片的节点，该芯片专门用于检测 Type-C 连接状态。

7.2. USB 工作模式介绍

执行如下命令查看 USB1 接口当前的工作模式。

```

root@quectel-srg093xw:~# cat /sys/bus/platform/drivers/ci_hdrc/ci_hdrc.0/role
gadget
root@quectel-srg093xw:~# cat /sys/bus/platform/drivers/ci_hdrc/ci_hdrc.0/role
host

```

插入 USB 闪存盘至 EVB 的 USB1 接口后，将显示类似以下内容的输出。

```

ci_hdrc ci_hdrc.0: EHCI Host Controller
ci_hdrc ci_hdrc.0: new USB bus registered, assigned bus number 2
ci_hdrc ci_hdrc.0: USB 2.0 started, EHCI 1.00
hub 2-0:1.0: USB hub found
hub 2-0:1.0: 1 port detected
usb 2-1: new high-speed USB device number 2 using ci_hdrc
usb-storage 2-1:1.0: USB Mass Storage device detected
scsi host0: usb-storage 2-1:1.0
scsi 0:0:0:0: Direct-Access   Generic  MassStorageClass 1538 PQ: 0 ANSI: 6
sd 0:0:0:0: [sda] Media removed, stopped polling
sd 0:0:0:0: [sda] Attached SCSI removable disk
scsi 0:0:0:1: Direct-Access   Generic  MassStorageClass 1538 PQ: 0 ANSI: 6
sd 0:0:0:1: [sdb] 62357504 512-byte logical blocks: (31.9 GB/29.7 GiB)
sd 0:0:0:1: [sdb] Write Protect is off

```

```
sd 0:0:0:1: [sdb] Mode Sense: 21 00 00 00
sd 0:0:0:1: [sdb] Write cache: disabled, read cache: enabled, doesn't support DPO or
sdb: sdb1
sd 0:0:0:1: [sdb] Attached SCSI removable disk
```

执行 **lsusb**，查看连接到 USB 端口的设备。

```
root@quectel-srg093xw:~# lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 002: ID 05e3:0749 Genesys Logic, Inc. SD Card Reader and Writer
```

8 ADC

SRG091X 系列和 SRG093X 系列模块支持 8 通道 12 位 ADC，其中低 4 路通道（CH0–CH3）已引出至模块外部引脚。

8.1. 设备树配置

如下方设备树所示，根据硬件设计配置 `vref-supply` 属性的参考电压，并将 `status` 属性设置为 “okay”，即可启用 ADC1。

```
/ {
    reg_vref_1v8: regulator-adc-vref {
        compatible = "regulator-fixed";
        regulator-name = "vref_1v8";
        regulator-min-microvolt = <1800000>;
        regulator-max-microvolt = <1800000>;
    };
}

&adc1 {
    vref-supply = <&reg_vref_1v8>;
    status = "okay";
};
```

8.2. 获取 ADC 电压

执行以下命令获取 ADC 转换比例系数。

```
root@quectel-srg093xw:~# cat /sys/bus/iio/devices/iio\:device0/in_voltage_scale
0.439453125
```

将 ADC 的 CH0 和 CH3 对应引脚连接到稳定的 1.8 V 电源，将 CH1 和 CH2 对应引脚连接到 EVB 的 GND 引脚。然后执行以下命令，获取各通道的数字取值。

```
root@quectel-srg093xw:~# cat /sys/bus/iio/devices/iio\:device0/in_voltage0_raw
4076
```

```
root@quectel-srg093xw:~# cat /sys/bus/iio/devices/iio\:device0/in_voltage1_raw
269

root@quectel-srg093xw:~# cat /sys/bus/iio/devices/iio\:device0/in_voltage2_raw
194

root@quectel-srg093xw:~# cat /sys/bus/iio/devices/iio\:device0/in_voltage3_raw
4081
```

实际电压的计算方法：实际电压 = in_voltage_raw × in_voltage_scale（单位：mV）。例如，CH0 的实际电压为：4076 × 0.439453125 = 1791 mV，约为 1.8 V。

9 CAN

i.MX 9 平台集成了两个 CAN FD 控制器，使用时需要外接收发器芯片。测试 CAN 数据收发功能需要使用两块开发板：开发板 A 用于数据发送，开发板 B 用于数据接收。

步骤一： 在开发板 A 和开发板 B 上执行以下命令，配置 CAN 接口。

- 1) 检查 can0 接口是否就绪。如果返回以下粗体所示的信息，则表示 can0 接口已就绪。

```
root@quectel-srg093xw:~# ip link show can0
4: can0: <NOARP,ECHO> mtu 16 qdisc noop state DOWN mode DEFAULT group default qlen 10
   link/can
```

- 2) 将 can0 接口的比特率配置为 500000 bps。

```
root@quectel-srg093xw:~# ip link set can0 type can bitrate 500000
```

- 3) 执行以下命令启用该接口。

```
root@quectel-srg093xw:~# ip link set up can0
```

- 4) 执行 **ip link show can0**，检查 can0 接口是否配置成功。如果返回以下粗体所示的信息，则表示 can0 接口已配置成功并处于就绪状态。

```
root@quectel-srg093xw:~# ip link show can0
4: can0: <NOARP,UP,LOWER_UP,ECHO> mtu 16 qdisc pfifo_fast state UP mode DEFAULT group default qlen 10
   link/can
```

步骤二： 使用 can-utils 测试 CAN 通信。

- 1) 在开发板 A 上执行以下命令发送数据。

```
root@quectel-srg093xw:~# cansend can0 123#11223344
```

- 2) 在开发板 B 上执行 **candump can0** 监听数据。如果通信正常，终端中将显示一条 ID 为 123、数据为 11223344 的 CAN 报文。

```
root@quectel-srg093xw:~# candump can0
can0 123 [4] 11 22 33 44
```

10 附录 术语缩写

表 2：术语缩写

缩写	英文全称	中文全称
ADC	Analog-to-Digital Converter	模数转换器
AP	Access Point	接入点
CAN	Controller Area Network	控制器局域网
EVB	Evaluation Board	评估板
GPIO	General Purpose Input/Output	通用型输入/输出
I2C	Inter-Integrated Circuit	集成电路总线
ID	Identifier	标识符
MISO	Master In Slave Out	主机输入从机输出
MOSI	Master Out Slave In	主机输出从机输入
OTG	On The Go	即插即用
PC	Personal Computer	个人电脑
SMBus	System Management Bus	系统管理总线
SPI	Serial Peripheral Interface	串行外设接口
USB	Universal Serial Bus	通用串行总线